Webpack 2: The Complete Developer's Guide

Section 1: How to Use This Course

Start Here!

1. https://github.com/StephenGricer
    1. WebpackProject

Course Resources

1. Completed Code

Join Our Community!

1. https://discord.gg/h2G3CbxPZA

Section 2: What Does Webpack Do?

Why Do We Use Build Tools?

1. What is Webpack?
2. Why use Webpack?
    1. Rich web applications (dynamic features)
        1. Server Side Templating
            1. Back end server constructs on HTML document and sends it to the user
                1. Fully rendered HTML document - all info exists in it already
            2. Server does it.
            3. Flow:
                1. User visits page.
                2. HTTP request to server.
                3. New HTML Document.
                4. User clicks a link.
                5. HTTP request to server.
                6. New HTML Document.
        2. Single Page Apps

1. Server sends a bare-bones HTML doc to the user. Javascript runs on the users machine to assemble a full webpage.
2. JS does it on user's machine.
3. Flow:
    1. User visits page
    2. HTTP request to server
    3. New HTML Document (+ JS scripts)
    4. React/Angular boots up, shows page
        1. Assembes some HTML to display.
    5. User clicks link
    6. React/Angular shows new content
        1. Doesn't load HTML
2. JS code assembles application in SPA
    1. JS is shipped to Browser

Javascript Modules

1. What are the problems with a lot of JS?
    1. Example: 10s of thousands in main.js, util.js, and home.js
        1. Go find header and change nav menu
            1. Only handful of files is a challenge.
                1. Hard to find and make changes
    2. The problem worsens with size of code
        1. Solution: JS modules - It becomes clear where the code is located
            1. components folder
                1. header.js
                2. tooltip.js
                3. dashboard.js
                4. button.js
                5. footer.js
                6. select.js
                7. dropdown.js
                8. range.js
                9. searchbar.js
                10. radiobutton.js
            2. API folder
                1. fetch.js
                2. keys.js
                3. formatter.js
                4. collection.js
            3. Too many small files is also challenging to handle

The Purpose of Webpack

1. Problems with separating code into files
    1. Load Order - modules must be loaded in a specific order
        1. Example:
            1. index.js
                1. store.js (depends on the following)
                    1. api.js
                2. util.js
    2. Load Time:
        1. If there a lot of files, the downloads will be slow
        2. Big issue on a mobile connection
        3. Fewer request will result in fewer connections
2. Webpack function: Takes big pile of JS files and merge them together
    1. Input files -> Webpack -> bundle.js
        1. Order is maintained
        2. Bundles into a single file

Webpack in Action

1. Project
    1. Tasks:
        1. Make a new NPM project in the terminal
            1. NPM modules as dependencies - Webpack
        2. Construct two JS modules (files)
            1. Module system
        3. Install and configure webpack
            1. Min config required to work properly
        4. Run webpack
            1. What it does.
2. Terminal:

```
mkdir js_modules
cd js_modules
npm init # continue
code .
```

3. VS Code:
    1. New folder: `src`
        1. `index.js`

2. `sum.js`

Javascript Module Syntax

1. index.js
    1. Calls functions from sum.js, then prints the result
2. sum.js
    1. Utility functions for working with mathematical calculations
3. index.js depends upon sum.js (needs to be loaded first)
4. sum.js

```
const sum = (a, b) => a + b;
```

1. Each module (file) has its own scope
    1. It is not inherently accessible in other modules (files)
        1. We need explicit link

5. Module systems in use:
    1. CommonJS - NodeJS (outside browser)
        1. `module.exports` **(M)**
        2. `require` **(M)**
    2. AMD (Asynchronous Module Definition) - Frontend apps (for loading modules asynchronously)
        1. `define` **(M)**
        2. `require` **(M)**
    3. ES2015 - Modern Web development
        1. `export` **(M)**
        2. `import` **(M)**
    4. ...

Linking Files with CommonJS

1. sum.js
    1. I have no dependencies
    2. Code
    3. This part of me is public: `sum` function (code provided to others)
        1. `module.exports = sum;`
2. index.js
    1. I need sum.js

1. `const sum = require("./sum");` - relative path reference (`.` current directory)
2. Code
3. I have no exports

Webpack Installation and Configuration

1. Terminal:
    1. `npm install --save-dev webpack@2.2.0-rc.0`
2. Configuration:
    1. `webpack.config.js` (name is important)

```
const path = require("path"); // handled by Node.js

const config = {
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "build"), // `__dirname` is a globa
    filename: "bundle.js",
  },
};

module.exports = config;
```

1. `entry` property - entry file
    1. `index.js` - bootstrap file - root (entry point)
        1. First file to execute
    2. It is the starting point of loading process
2. `output` - where to save the bundle and what should its name be
    1. `bundle.js` - name can be anything
    2. `path` - needs to be absolute path (not relative path)

Running Webpack

1. `package.json`

```
"scripts": {
  "build": "webpack"
}
```

1. Why script?
    1. webpack: Out of all globally installed modules, find the one called webpack and run it (`-g`)
        1. Bad approach
    2. `npm run build` - Run the copy of 'webpack' stored in the 'node_modules' directory
        1. Pros:
            1. We can only one version and it will be compatible. (each project owns it)
                1. We can depend on locally installed version.

2. Running webpack:

```
npm run build
```

3. `bundle.js` is larger than the sum of the files

The Bundle.js File

1. `bundle.js` pseudocode:

```
var myModules = [
  function () {
    const sum = (a, b) => a + b;
    return sum;
  },
  function () {
    const sum = myModules[0]();
    const total = sum(10, 10);
    console.log(total);
  }
];

var entryPointIndex = 1;
myModules[entryPointIndex]();
```

1. First function wraps whatever is in `sum.js`
2. Second funciton wraps whatever is in `index.js`
3. `entryPointIndex` - index of the function designated as an entry point
4. It is vanela JS code

2. Complex cases handled by `bundle.js`
    1. Handling compatibility between different module systems
    2. Caching of modules
        1. Avoids repeatedly calling module functions again and again.

Running the App

1. HTML file to load up bundle.js
    1. `index.html` - Webpack convention

```
<head>
  <title>Webpack</title>
</head>

<body>
  <h1>Webpack</h1>
  <script src="build/bundle.js"></script>
</body>
```

Section 3: Handling Project Assets

Introduction to Loaders

1. Additional capabilities:
    1. Module loaders
        1. Pre-processing of files before adding to bundle.js
        2. Example:
            1. Transpiling
            2. Handle CSS
            3. Handle images
            4. ...
2. Loaders
    1. Babel - Transpiles to ES5 code
        1. Turn ES2015 (ES7, ...) code into ES5 code
        2. 3 modules to get babble working
            1. Module:
                1. babel-loader - Teaches babel how to work with webpack

        1. compatibility layer
    2. babel-core - Knows how to take in code, parse it, and generate some output files
        1. It doesn't actually transpile
    3. babel-preset-env - Ruleset for telling babel exactly what pieces of ES2015/6/7 syntax to look for, and how to turn it into ES5 code
        1. Has rules
            1. Arrow functions to functions
            2. Class to functions
            3. let to var
            4. ...
3. Installation:

```
npm install --save-dev babel-loader babel-core babel-preset-env
```

3. Workflow:
    1. Input: ImageViewer.js, index.js
        1. Add it to webpack config
        2. Tell webpack on which files the loader needs to be applied
            1. We don't want Babel to operate on css files (say) - results in error
4. webpack.config.js

Babel Setup for ES2015

1. webpack.config.js

```
const path = require("path"); // handled by Node.js

const config = {
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "build"), // `__dirname` is a global va
    filename: "bundle.js",
  },
  module: {
    rules: [
      {
        use: "babel-loader",
        test: /\.js$/, // test: /\.js$/ means that the babel-loader will
      },
    ],
```

```
    },
  };

  module.exports = config;
```

2. `.babelrc`

```
{
  "presets": ["babel-preset-env"] // This is a preset that includes all the necessary plugins for the latest JavaScript features. Babel will use this preset to transpile the code.
}
```

3. Fix:

```
npm uninstall --save-dev webpack
npm uninstall --save-dev babel-loader babel-core babel-preset-env

npm install --save-dev webpack
npm install --save-dev babel-loader @babel/core @babel/preset-env
```

1. `.babelrc`

```
{
  presets: ["@babel/preset-env"]
}
```

2. `package.json`

```json
"scripts": {
  "build": "webpack --mode development"
}
```

Babel Configuration

Refactor to ES2015 Modules

1. ES2015 - refactor from common js to es2015
2. Syntax:

```
import sum from './sum';
// ...

export default sum;
```

3. Fix:
    1. Remove comments from `.babelrc`
    2. Remove `"type": "commonjs" from` package.json` file
    3. Clear cache: `rm -rf node_modules/.cache`
    4. Rebuild: `npm run build`

Handling CSS with Webpack

1. Must use loaders:
    1. style loader
    2. css loader
2. We can use multiple css files and import them
    1. Example: `image_viewer.css` can be imported into `image_viewer.js` file
        1. It doesn't scope it to the file - it is only convenient to the developers
3. Two files: image_viewer.js, image_viewer.css
    1. src/image_viewer.js

```
const image = document.createElement("img");
image.src = "https://placehold.co/400x400";

document.body.appendChild(image);
```

    2. src/index.js

```
import sum from "./sum";
import "./image_viewer"; // The file does not export anything, so we

const total = sum(5, 10);

console.log(total);
```

The Style and CSS Loaders

1. Importing a css file:
    1. styles/image_viewer.css

```
img {
  border: 10x solid black;
}
```

    2. src/image_viewer.js

```
import '../styles/image_viewer.css'; // File extension is required for
non-js files

// ...
```

2. webpack.config.js

```
const path = require("path"); // handled by Node.js

const config = {
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "build"), // `__dirname` is a global va
    filename: "bundle.js",
  },
  module: {
    rules: [
      {
        use: "babel-loader",
        test: /\.js$/, // test: /\.js$/ means that the babel-loader will
      },
      {
        use: ["style-loader", "css-loader"], // style-loader is used to i
        test: /\.css$/,
      },
    ],
  },
};

module.exports = config;
```

2. New modules:
    1. css-loader

1. To teach webpack how to import and parse css files.
    1. To just open a css file and read its contents
2. style-loader
    1. Takes css imports and adds them to the HTML document (in a style tag).
        1. To teach webpack what to do with the content
3. Installation:

```
npm install --save-dev style-loader css-loader
```

Loaders are Tricky!

1. Where is the CSS?
    1. Dev Tools

```
<head>
  <style>...</style>
</head>
```

        1. bundle.js
            1. Search: `border`
                1. CSS injected as raw string (single line)
                2. Style loader takes modules, and manually injects CSS a

The Extract Text Plugin

1. Separate CSS file - Browser can download it in parallel
2. Installation:

```
npm install --save-dev extract-text-webpack-plugin
```

1. It takes a reference to a loader, it will run webpack with it, and takes text generated into a separate file.
        1. Fix for new versions of Webpack:

        ```bash
        npm install --save-dev mini-css-extract-plugin
        ```

            1. webpack.config.js

```javascript
const MiniCssExtractPlugin = require("mini-css-extract-plugin");
const path = require("path"); // handled by Node.js

const config = {
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "build"), // `__dirname` is a global variable that refers to the directory name of the current module. Handled by Node.js.
    filename: "bundle.js",
  },
  module: {
    rules: [
      {
        use: "babel-loader",
        test: /\.js$/, // test: /\.js$/ means that the babel-loader will be applied to all files with the .js extension.
      },
      {
        use: [MiniCssExtractPlugin.loader, "css-loader"], // style-loader is used to inject the CSS into the DOM. css-loader is used to resolve the CSS files. Loaders are applied in reverse order.
        test: /\.css$/,
      },
    ],
  },
  plugins: [
    new MiniCssExtractPlugin({
      filename: "style.css",
    }),
  ],
};

module.exports = config;
```

1. Loaders - used for pre-processing before the file is included into the webpack bundle

2. Plugins - work outside webpack. They keep the files outside webpack bundle

1. `plugins` - Anything a `MiniCSSExtractPlugin.loader` has caught, is passed to the plugin object.

Handling Images with Webpack

1. It takes long to load the image.
    1. We can download and serve it locally
    2. Handling images:

```
Image
| resize the image
v
image-webpack-loader (compresses image to reduce file size) - part of the pipeline
| is the image small?
v
url-loader (behavior is different based on size)
| is the image small?     | is the image big?
v                         v
Include the image         Include the raw image
in bundle.js              in the output directory
as raw data
(some performance gain)
```

2. Installation:

```
npm install --save-dev image-webpack-loader url-loader
```

3. Configuration:

```
const MiniCssExtractPlugin = require("mini-css-extract-plugin");
const path = require("path"); // handled by Node.js

const config = {
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "build"), // `__dirname` is a global va
    filename: "bundle.js",
  },
  module: {
```

```
    rules: [
      {
        use: "babel-loader",
        test: /\.js$/, // test: /\.js$/ means that the babel-loader will
      },
      {
        use: [MiniCssExtractPlugin.loader, "css-loader"], // style-loader
        test: /\.css$/,
      },
      {
        use: [
          {
            loader: "url-loader",
            options: {
              limit: 40000, // 40kb large
            },
          },
          "image-webpack-loader",
        ],
        test: /\.(jpe?g|png|gif|svg)$/,
      },
    ],
  },
  plugins: [
    new MiniCssExtractPlugin({
      filename: "style.css",
    }),
  ],
};

module.exports = config;
```

Automatic Image Compression

1. Download images: https://picsum.photos/1200/1200, https://picsum.photos/200/200
2. Fix:

```
npm install --save-dev file-loader
```

Public Paths

1. webpack.config.js

```
const config = {
  entry: "./src/index.js",
  output: {
    ...
    publicPath: "build/",
  },
```

1. Dev Folder - Image -> Build Folder
    1. URL Loader emits the URL of the file, with `output.publicPath` prepended to the URL

Section 4: Building for Performance with Webpack

Introduction to Code Splitting

1. Code-splitting:
    1. Two screens say
        1. Login screen
            1. Has a little bit of JS code
            2. Only load minimum amount of JS necessary to show a login form
                1. To minimize network resource usage
        2. Dashboard - charts and graphs
            1. Several thousand lines of JS code
            2. Logged? Ok, now load the rest of the JS
    2. It allows us to split bundle.js into separate individual files and programmatically decide when to load different pieces of the bundle
        1. Example: When user opens login screen, load only the JS required for the login screen.
        2. Disadvantage: A delay is introduced between pages.

Codesplitting in Practice

1. When user clicks, an image is shown.
2. Code-splitting:
    1. index.js
        1. Entry point. Render a button with click handler. Imports 'image_viewer' after button gets clicked.
    2. image_viewer.js - Renders an image
3. `System.import('./image_viewer');`

```
Client                                          Server
|
button clicked
|
System.import call              ->      Fetch image_viewer module
|
Execute image_viewer    <-      Reply with image_viewer
```

On Demand Code Loading

System Import Calls

1. index.js

```
const button = document.createElement("button");
button.innerText = "Click me";
button.onclick = () => {
  import("./image_viewer").then((module) => {
    console.log(module);
  }); // Special function of ES2015 spec. Browser reaches server and trie
};

document.body.appendChild(button);
```

2. image_viewer.js

```
import small from "../assets/small.jpg"; // base64 string of the image
import "../styles/image_viewer.css";

export default () => {
  const image = document.createElement("img");
  image.src = small;

  document.body.appendChild(image);
};
```

1. Webpack sees `import` and add the additional code

Section 5: Applying Webpack to a Real Project

A Real World Project

1. Project: github.com/StephenGrider/WebpackProject
    1. Clone it.
2. `react` transpiles JSX to JS.

Setting Up Babel

Minimum Webpack Config

1. Latest working application: https://github.com/bmutthoju/my-app

Vendor Asset Caching

1. Optimization with webpack doesn't impact runtime performance
    1. It helps is speeding up load time (smaller size files)
2. Splitting code-base using code-splitting
    1. Our code:
        1. Index.js
        2. SearchList.js
    2. Vendor code:
        1. React.js
        2. Lodash.js
        3. ...
3. Browser caching
    1. First visist: file_a.js -> Have I downloaded this before? -> No! -> Ok, download it.
    2. Next visit: file_a.js -> Have I downloaded this before? -> Yes! -> Great. I'll download nothing!
4. We can exploit the browser caching to improve load time.

More on Vendor Caching

1. Updates to our code happens more frequently than vendor code
    1. Code splitting helps with load time
        1. `vendor.js` - all dependencies of the project
            1. `bundle.js` - downloaded more frequently
2. webpack.config.js

```
// Node.js built-in module to handle file paths
const path = require("path");
```

```
// Plugin to generate an HTML file and inject script tags automatically
const HtmlWebpackPlugin = require("html-webpack-plugin");

module.exports = {
  // Entry point(s) of the application
  entry: {
    bundle: "./src/index.js", // main React app entry point
  },

  // Output configuration for bundled files
  output: {
    path: path.resolve(__dirname, "dist"), // folder to output the bundle
    filename: "[name].[contenthash].js", // [name] = entry name, contenth
    clean: true, // cleans the dist folder before each build
  },

  // Module rules specify how different file types are processed
  module: {
    rules: [
      {
        test: /\.js$|\.jsx$/, // matches .js and .jsx files
        exclude: /node_modules/, // ignore node_modules
        use: "babel-loader", // transpile ES6+ and JSX into browser-compa
      },
      {
        test: /\.css$/i, // matches .css files
        use: ["style-loader", "css-loader"],
        // css-loader: interprets @import and url() like import/require
        // style-loader: injects CSS into the DOM
      },
      {
        test: /\.(png|jpe?g|gif|svg)$/i, // matches common image types
        type: "asset/resource", // emits images as separate files and ret
      },
    ],
  },

  // Automatically resolves these extensions so imports don't need extens
  resolve: {
    extensions: [".js", ".jsx"], // import files without specifying .js c
  },

  // Plugins add extra functionality to Webpack
  plugins: [
```

```
    new HtmlWebpackPlugin({
      template: "./public/index.html", // base HTML file to inject script
    }),
  ],

  // Optimization configuration for code splitting and caching
  optimization: {
    splitChunks: {
      chunks: "all", // consider all types of chunks (sync + async)
      cacheGroups: {
        defaultVendors: {
          test: /[\\/]node_modules[\\/]/, // target all modules from node
          name: "vendor", // name of the resulting vendor chunk
          chunks: "all", // apply to all chunks
          priority: -10, // lower priority than other cacheGroups if defi
          reuseExistingChunk: true, // reuse existing chunk if identical
        },
        default: {
          minChunks: 2, // modules must be shared by at least 2 chunks
          priority: -20, // lower priority than vendors
          reuseExistingChunk: true, // reuse chunks if possible
        },
      },
    },
  },

  // Development server configuration
  devServer: {
    static: "./dist", // serve files from dist folder
    hot: true, // enable Hot Module Replacement
    port: 3000, // run dev server on port 3000
  },
};
```

Refactoring for Vendor Splitting

Effect of Code Splitting

Troubleshooting Vendor Bundles

1. Something not working.
    1. index.html

1. We need to add `vendor.js`
    1. Solution: `html-webpack-plugin`

```
// Plugin to generate an HTML file and inject script tags au
const HtmlWebpackPlugin = require("html-webpack-plugin");
...
// Plugins add extra functionality to Webpack
plugins: [
  new HtmlWebpackPlugin({
    template: "./public/index.html", // base HTML file to in
  }),
],
```

Chunk Hashing for Cache Busting

1. Renaming files when contents change.
    1. webpack tweek:

```
output: {
  path: path.resolve(__dirname, "dist"), // folder to output the bunc
  filename: "[name].[contenthash].js", // [name] = entry name, conter
  clean: true, // cleans the dist folder before each build
},
```

Cache Busting Wrapup

1. Hash depends on content of the file
    1. Browser re-downloads the file if hash changed.

Cleaning Project Files

1. A new file is created with other files not cleaned up by default.
    1. `rimraf` - package to handle
        1. Handles inconsistencies between commands on Windows and OSX. (`rm` is not available on Windows)

```
"scripts": {
  "clean": "rimraf dist",
```

```
    "build": "npm run clean && webpack --mode production"
  },
```

Section 6: Webpack Dev Server

Introduction to Webpack Dev Server

1. Project directory
    1. index.html - first ->
    2. vendor.js - second ->
    3. bundle.js - third ->
2. `webpack-dev-server` - intermediary between webpack output and browser
    1. It watches project files and automatically rebuilds the project
        1. It updates only the changed files
    2. It automatically feeds us index.html
    3. SPA with no logic in server
        1. Doesn't support DB, Websockets, authentication, etc.

Gotchas with Webpack Dev Server

1. Installation
    1. `npm install --save-dev webpack-dev-server`
2. package.json

```
"scripts: {
  "serve": "webpack-dev-server"
}
```

3. `npm run serve`
    1. `npm run clean`
    2. `npm run serve` - `dist` was not created
        1. dev server stops webpack from saving on disk. They are only saved in memory.

Section 7: React-Specific Topics

React Router with Codesplitting

1. router.js

```
// src/router.js
import React from "react";
import { createBrowserRouter, RouterProvider } from "react-router-dom";
import Home from "./components/Home";
import About from "./components/About";

// Define your route configuration
const router = createBrowserRouter([
  {
    path: "/",
    element: <Home />,
  },
  {
    path: "/about",
    element: <About />,
  },
]);

// Wrap the RouterProvider
const AppRouter = () => {
  return <RouterProvider router={router} />;
};

export default AppRouter;
```

2. src/components/About.js

```
// src/components/About.js
import React from "react";
import { Link } from "react-router-dom";

const About = () => {
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>About Page</h1>
      <p>This app is bundled with Webpack and uses React Router v6!</p>
      <Link to="/">← Back to Home</Link>
    </div>
  );
};

export default About;
```

3. src/components/Home.js

```
// src/components/Home.js
import React from "react";
import { Link } from "react-router-dom";

const Home = () => {
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>Welcome to My React App</h1>
      <p>This is the Home page.</p>
      <Link to="/about">Go to About Page →</Link>
    </div>
  );
};

export default Home;
```

4. src/index.js

```
import React from "react";
import { createRoot } from "react-dom/client";
import AppRouter from "./router";
import "./style.css";

const container = document.getElementById("root");
const root = createRoot(container);

root.render(<AppRouter />);
```

Plain Routes with React Router Codesplitting

1. Webpack-dev-server serves only static assets (no db calls allowed)
    1. The entire app is running on browser
2. Static asset providers
    1. Github pages
    2. Amazon S3
    3. Digital Ocean
    4. MS Azure
    5. Surge - Easiest
3. Server-based providers

1. Amazon EC2
2. Amazon ELB
3. Digital Ocean
4. Heroku
5. MS Azure

Section 8: Webpack-Based Deployment for Static Sites

Deployment Options

1. Config for production based practices.
    1. React JS uses `NODE_ENV` flag. It looks for `process.env.NODE_ENV` in browser.
        1. If `production`, React doesn't do much error checking
        2. If `development`, React
2. `webpack --mode production` - compacts code and minifies.
    1. `NODE_ENV=production` is not required because webpack sets it based on the mode
    2. The following section is not required due to `mode`:

```
new webpack.DefinePlugin({
  // 👇 This replaces all instances of process.env.NODE_ENV in your c
  // with the actual environment string (e.g. "development" or "produ
  // This substitution happens at compile time, NOT at runtime.
  "process.env.NODE_ENV": JSON.stringify(process.env.NODE_ENV),
}),
```

Getting Production Ready

Deployment with Surge (Easiest!)

1. surge.sh - best static website publishing tool - if no backend
    1. `npm install -g surge` **(M)**
    2. `npm run build`
    3. `surge -p dist`
2. Adding a custom domain

Deployment with Github Pages

1. Special branch name: `gh-pages`
    1. Automatically served on: `https://<Username>.github.io/<RepoName>`
        1. Custom domain name is also supported.
    2. https://docs.github.com/en/pages/quickstart
    3. https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site#creating-your-site
2. Github pages:

```
git checkout -b gh-pages
git subtree push --prefix dist origin gh-pages # push only dist folder to remote gh-pages branch
```

Deployment with AWS S3

1. Library - `npm install -g s3-website`
    1. Automatically generates a bucket and deploys
    2. Needs API Keys
        1. Open aws
            1. Account is needed
            2. Name > My security credentials
                1. Access Keys
                    1. Create new access key - Download Key File
                        1. Don't commit them on Github
        2. New file under root: `.env`

        ```
        AWS_ACCESS_KEY_ID=<ID>
        AWS_SECRET_ACCESS_KEY=<SECRET>
        ```

    3. Bucket creation: `s3-website create webpack-deploy-bucket-bmutthoju`
        1. Bucket URL
    4. Deployment: `s3-website deploy dist` (can be repeated)
        1. Site URL
2. Cleanup:
    1. Delete access key
    2. Delete bucket

Section 9: Webpack-Based Deployment for Dynamic Sites

Deployment of Servers

1. Complex server
    1. Internal requests
    2. Connects to DB
    3. Authentication
    4. ...
2. Deployment structures
    1. Assets + Custom Server
        1. Built app (index.html, bundle.js, vendor.js) - I want to see the app (http://www.app.com) -> Browser
        2. Server - I want to make an API request (http://api.app.com) -> Browser (Built app makes API requests)
            1. For large applications - seperation of concerns
    2. Server responsible for both Built App and Backend functions (Node Server) - for smaller applications (deployment is easy)
        1. Built App - index.html, bundle.js, vendor.js
            1. I want to view the app -> Browser
        2. Route Handler - I want to login -> Browser
        3. Route Handler - I want to add a to-do -> Browser
            1. It is hard with Webpack
                1. Node takes over putting Webpack on the side
                    1. Node server must reach Webpack and serve application bundle

Node and Webpack Integration

1. Node server behaves differently depending on the envoronment.
2. Local machine
    1. Development Environment
        1. Built App
            1. index.html
            2. bundle.js
            3. vendor.js
        2. Node Server
            1. Build App - webpack middleware - I want to view the app -> Browser

1. When request comes to Node Server, it is forwarded to webpack process through webpack middleware
2. Production: We don't want to run webpack
    1. We want to build our application and save the generated assets in dist
    2. When user sends a request, the files are directly served from the directory
2. Route Handler - I want to login -> Browser
3. Route Handler - I want to add a to-do -> Browser

3. Install Express JS
    1. `npm install --save express`
4. server.js

```
const express = require("express");

const app = express();

app.listen(3050, () => console.log("Listening"));
```

Webpack Middleware in Development

1. Wiring up Webpack
    1. Middleware intercepts requests for application and responds with JS application (assets)
    2. Installation:
        1. `npm install --save-dev webpack-dev-middleware`
    3. server.js

```
...
const webpackMiddleware = require('webpack-dev-middleware'); // It or
const webpack = require('webpack');
const webpackConfig = require('./webpack.config.js');

const app = express();

app.use(webpackMiddleware(webpack(webpackConfig)));

app.listen(...);
```

1. Webpack runs in node server
2. On making a change and refreshing the page, the change gets reflected

Webpack Middleware in Production

1. Node server will fetch resources from `dist` directory and return them.
2. server.js

```
const express = require("express");
const path = require("path");

const app = express();

if (process.env.NODE_ENV !== "production") {
  const webpackMiddleware = require("webpack-dev-middleware");
  const webpack = require("webpack");
  const webpackConfig = require("./webpack.config");

  app.use(webpackMiddleware(webpack(webpackConfig)));
} else {
  app.use(express.static("dist")); // makes content in dist freely availa
  app.get(/.*/, (req, res) => {
    res.sendFile(path.resolve(__dirname, "dist", "index.html")); // for c
  });
}

app.listen(3050, () => console.log("Listening"));
```

3. How to extend router.js
    1. Add additional routes above the webpack configuration

```
app.get('/hello', (req, res) => res.send({ hi: 'there' }));
```

1. `*` is a fallback. Otherwise the other routes will not handle the requests.

Deployment to Heroku

1. Easiest and has free tier
2. `Procfile` - automatically run by Heroku
    1. To run commands to startup server
3. Heroku:
    1. Sign up
4. Install Heroku CLI

    1. `heroku auth:login`
5. Run the following from project directory:

```
heroku create # returns a link
git push heroku main`
```

6. Open the link: `heroku open`
7. To get output from server: `heroku logs`

Deployment to AWS

1. Elastic Beanstalk
    1. Encapsulates other services
        1. EC2
        2. RDS
        3. Load Balancers
        4. ...
    2. Use-case: If app might get popular over time, and we want to meet increased demand.
    3. Free tier is available
2. Install ELB CLI - https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/eb-cli3.html

```
git clone https://github.com/aws/aws-elastic-beanstalk-cli-setup.git
python .\aws-elastic-beanstalk-cli-setup\scripts\ebcli_installer.py
eb --version
```

More on AWS Deployment

1. ELB
    1. Dashboard
    2. Configuration
        1. Scaling - load details
        2. Default instance type
        3. Load Balancing
        4. Data Tier
            1. RDS
    3. Health
        1. Requests
        2. Monitoring
            1. CPU
            2. Latency

3. ...

Extras

Bonus!

1. Rust: The Complete Developer's Guide
2. ChatGPT and LangChain: The Complete Developer's Masterclass
3. Redis: The Complete Developer's Guide
4. Go: The Complete Developer's Guide (Golang)